

AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH (AIUB)

Faculty of Science and Technology (FST)

Department of Computer Science (CS)

Undergraduate Program

PROJECT REPORT

GameHub — Game Shop Management System

Course Code & Title	CSC 2210 — Object Oriented Programming 2
Credit	3 Credits (2 hrs Theory + 2 hrs 20 min Lab / week)
Nature	Core Course for CSE and COE
Prerequisite	CSC 2209 — Object Oriented Programming 1
Semester	Spring 2025-2026
Application Type	Desktop Management Application (C# WinForms)
Database	Microsoft SQL Server (Relational, 3NF)

Group Members

Sl. No.	Name	ID	Contribution
1	Tanvir Islam Tusar	24-57567-2	25%
2	Mahfuz Ahammed Rizvi	24-57543-2	25%
3	Shihab Ahmed	24-59539-3	25%
4	Mohammad Salman Farook	24-57575-2	25%

Table of Contents

1. Project Title	3
2. Project Description	3
3. Subscription Plans.....	4
4. Unique & Value-Adding Features.....	5
5. General Requirements.....	7
6. Proposed Class Design	9
7. Database Schema Diagram.....	10

8. Database Implementation (SQL Queries)	13
9. Navigation Structure.....	20
10. Output Screenshots	22
11. Tools & Technologies.....	31
12. Additional Notes	31

1. Project Title

GameHub — A Game Shop Management System.

GameHub is a desktop-based game retail management application, inspired by digital storefronts such as Steam, that lets a game shop manage its game catalogue, inventory, customers, orders, billing, and sales reporting from a single Windows application. The project is built in C# (Windows Forms) with a Microsoft SQL Server relational database and demonstrates the four core Object-Oriented Programming principles throughout its class design.

2. Project Description

The system supports three types of users — Admin, Staff, and Customer — each with a dedicated set of functionalities. After logging in with an ID and password, users are routed to the interface appropriate to their role. The application manages the complete lifecycle of a game sale: cataloguing games, tracking stock, letting customers browse and purchase games, generating invoices, recording payments, and producing sales/revenue reports for management.

Business context: A game shop needs to keep its catalogue and stock accurate, let customers buy games quickly, and give managers a clear view of revenue. GameHub brings all of this into one maintainable, object-oriented desktop system.

User Roles and Features

User Type 1 — Admin

- Full CRUD control over the game catalogue (add, view, update, delete games).
- Set and update game prices, discounts, and stock levels.
- Create, edit, activate, and deactivate Staff and Customer accounts (user management).
- Manage genres, publishers, and categories.
- Process payments and refunds; reconcile order totals.
- View full sales and revenue reports, best-selling games, and top genres.
- Search across games, users, and orders.

User Type 2 — Staff / Moderator

- Add new games and edit game details (title, description, genre, publisher, cover, stock).
- Mark games as Featured or On-Sale.
- View and assist with the order list for customer support.
- Update inventory / restock quantities.
- Search the catalogue and orders.
- Cannot set prices, issue refunds, view revenue reports, permanently delete games, or manage Admin/Staff accounts.

User Type 3 — Customer

- Register a new account and log in.
- Browse the store grid and search / filter games by genre or title.
- View a detailed game page (description, price, publisher).
- Add games to a cart and check out with a simulated payment.
- Receive a printable invoice / bill for each purchase.

- View personal order history and owned-games library, and edit own profile.
- Cannot access inventory, pricing controls, or other users' data.

3. Subscription Plans

In addition to one-off game purchases, GameHub offers optional membership tiers (similar to Xbox Game Pass or PlayStation Plus). A customer can subscribe to a plan for recurring benefits such as purchase discounts, free monthly games, and early access to sales. Subscriptions are managed by the Admin and give the shop a steady, recurring source of revenue.

3.1 Membership Tiers

Plan	Monthly Price	Discount	Free Games / mo	Key Benefits
Basic	Tk 199	5%	0	Wishlist price alerts, sale-event access, standard support
Plus	Tk 399	10%	1	Early access to pre-orders, 2x loyalty points, priority support
Ultimate	Tk 699	15%	2	Exclusive beta access, 3x loyalty points, gift game keys, 24/7 support

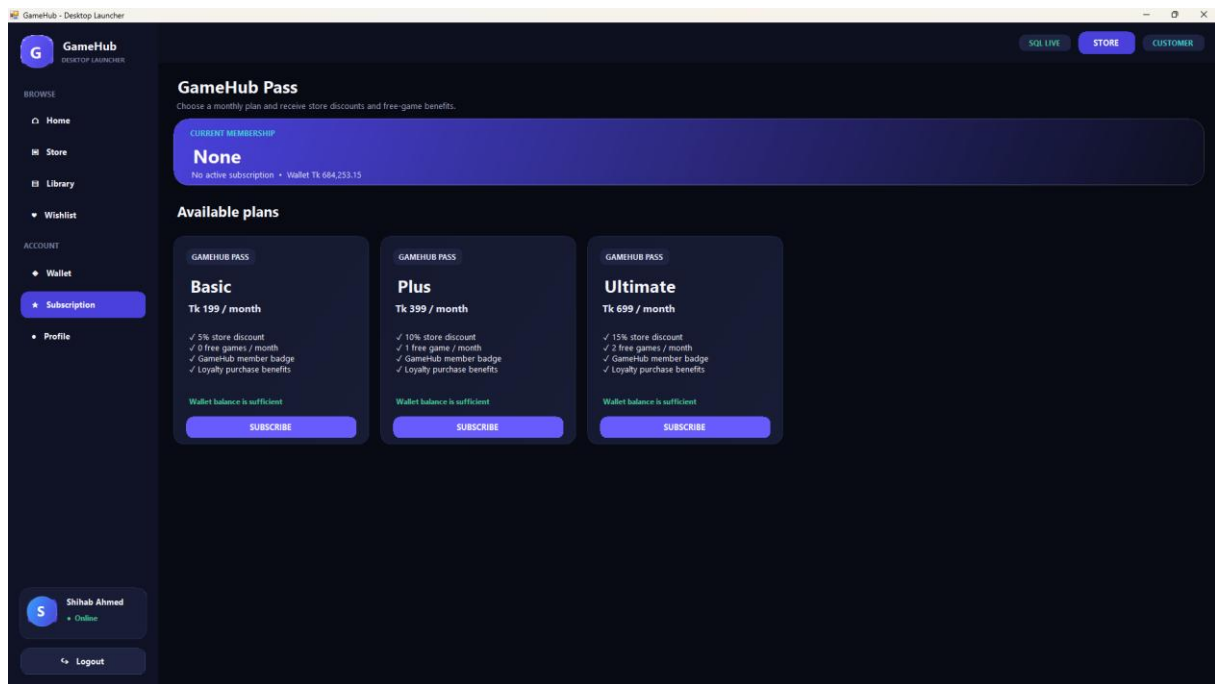


Figure 1: Subscription plans screen with tiered membership cards

3.2 How It Works

- Each customer's active plan is stored in a Subscriptions table linked to a SubscriptionPlans table (see the extended schema in Section 7).
- At checkout, the system automatically applies the member's discount to the order total.
- Every subscription has an expiry date (EndDate); the app checks it on login and marks expired plans inactive.
- Free monthly games are added directly to the member's owned-games library.
- Admin can view all active subscribers, total subscription revenue, and upgrade, downgrade, or cancel a membership.

OOP note: an abstract `ApplyDiscount()` method on a base `Plan` class is overridden by each tier (Polymorphism), so a new plan can be added later without changing the checkout code.

4. Unique & Value-Adding Features

Beyond the core course requirements, GameHub proposes several distinctive features that make it stand out from a basic management system. Each feature is backed by its own class and database table, so it also strengthens the OOP and database design. Features are tagged Core (planned for this version) or Bonus (optional, for extra credit).

Feature	What it does	Scope
Digital Wallet & Top-Up	Customers keep a prepaid balance and pay instantly without re-entering card details.	Bonus
Loyalty Reward Points	Earn points on every purchase and redeem them for discounts - gamifies buying.	Bonus
Ratings & Reviews	Customers rate games 1-5 and leave comments; the average rating shows on each game.	Core
Wishlist & Price Alerts	Save games and get notified when the price drops or the game goes on sale.	Core
Game Bundles / Combos	Admin groups several games into a single discounted bundle.	Bonus
Gift a Game	Buy a game and send it straight to another user's library.	Bonus
Pre-Order System	Reserve upcoming titles before release; auto-delivered on launch day.	Bonus
Promo Codes / Gift Cards	Redeemable codes give a discount or add wallet credit.	Bonus
Low-Stock Alerts & Analytics	Staff/Admin get low-stock alerts; the dashboard shows sales-trend charts.	Core
Dark / Light Theme	A theme toggle for comfortable viewing.	Bonus

4.1 Highlighted Features

Digital Wallet - users top up a balance (via a simulated bKash/card gateway) and spend it in one click. Every credit and debit is logged in a `WalletTransactions` table, so the balance is always auditable.

Loyalty Rewards - customers earn 10 points per Tk 100 spent; 1,000 points can be redeemed for a Tk 50 discount. This encourages repeat purchases and rewards loyal buyers.

Ratings & Reviews - buyers rate a game 1-5 and write a short review, stored in a `Reviews` table. The store page shows the average rating, helping new customers decide.

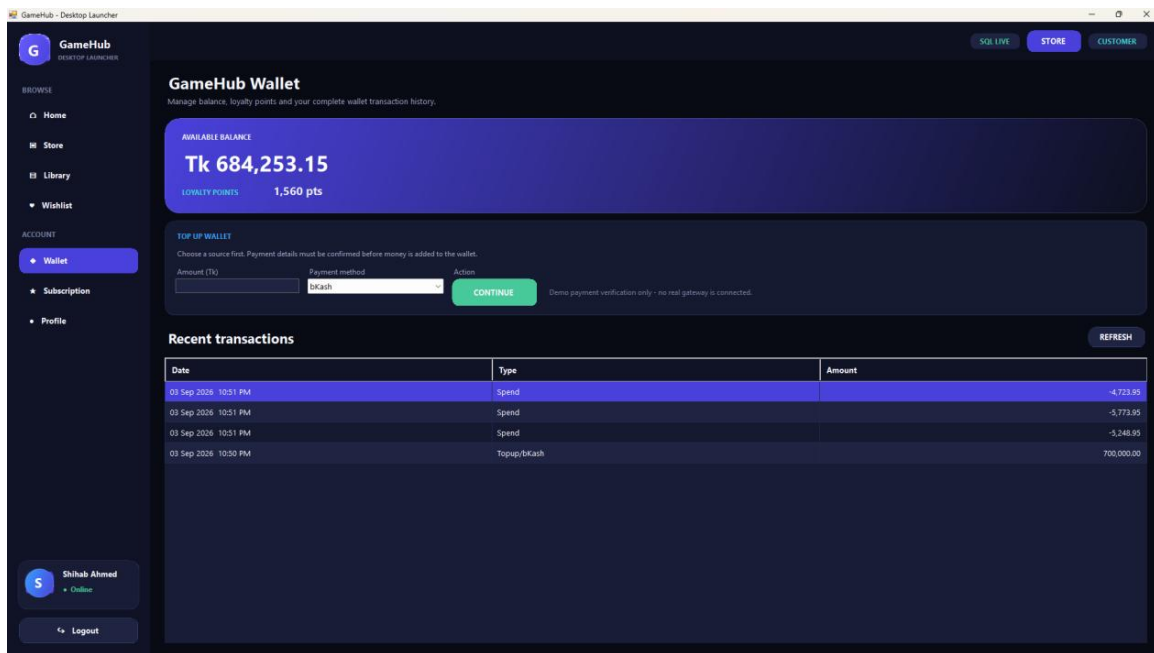


Figure 2: Wallet & loyalty-rewards screen with transaction history

5. General Requirements

5.1 System Design & Architecture

The application follows all four Object-Oriented Programming (OOP) principles:

Encapsulation — every entity (User, Game, Order, Payment) exposes its data through properties with private backing fields; internal state cannot be modified directly.

Inheritance — an abstract base class User is inherited by Admin, Staff, and Customer classes, which reuse shared members (ID, name, credentials).

Polymorphism — a virtual/abstract method such as GetDashboardAccess() (or ShowMenu()) is overridden by each user subclass to give role-specific behaviour.

Abstraction — the abstract User class and a DatabaseConnection class hide implementation details behind clean, simple interfaces.

- The system is a desktop-based application built in C# (Windows Forms).
- Proper class design is implemented with more than two user classes (Admin, Staff, Customer) plus entity and service classes.

Overall the project uses a simple layered architecture: Presentation layer (WinForms) → Business/Logic layer (entity & service classes) → Data Access layer (a single DatabaseConnection class using ADO.NET) → SQL Server database.

5.2 Database Requirements

- Uses a relational database — Microsoft SQL Server.
- The schema is normalized to at least Third Normal Form (3NF): no repeating groups, no partial dependencies, and no transitive dependencies (see the Schema Diagram in Section 5).
- Full CRUD operations (Create, Read, Update, Delete) are implemented on Games, Users, Orders, and Payments.
- A dedicated DatabaseConnection class centralizes the connection string and command execution, so no form talks to the database directly.

5.3 Functional Requirements

- A search option is available to all users (search games by title or genre).
- Login functionality requires only an ID and a Password.
- All forms are properly connected — navigation flows from Login to the correct role dashboard and between all management forms.
- A billing / record system generates an invoice for each order, applies VAT, records the payment, and stores the order in the database.

5.4 User Interface Requirements

- The application includes multiple forms: Login, Registration, Admin Dashboard, Manage Games, Inventory, Store/Browse, Cart & Billing, Users, and Reports.
- Proper form-design principles are applied: clear layout, consistent spacing, and user-friendly navigation (sidebar / menu based).
- Desktop UI design consistency is maintained across all forms (same colour theme, fonts, and control styles).

5.5 Validation & Error Handling

- Form validation checks required fields, numeric formats (price, stock), and valid email format on registration.
- Empty logins, duplicate usernames, and invalid quantities are blocked with clear messages.
- Exception handling (try-catch) wraps all database operations; any error is caught and shown to the user instead of crashing the application.

5.6 Programming Practices

- Proper use of properties (get/set) for all entity fields.
- Correct use of access modifiers: private fields, public properties/methods, and protected members in the base User class.
- Code is modular and maintainable — separated into entity classes, service/logic classes, a data-access class, and form (UI) classes.

5.7 Application Type

GameHub is a management-type application (a Game Shop / Inventory & Sales Management System), in the same family as the Library, Student, Restaurant, and Inventory management examples listed in the course requirements.

6. Proposed Class Design

The table below summarizes the main classes and how each contributes to the OOP requirements.

Class	Type	Responsibility / OOP role
User	Abstract base	Base for all users; holds Id, Name, credentials — Abstraction & Inheritance
Admin / Staff / Customer	Derived	Inherit from User; override role behaviour — Inheritance & Polymorphism
Game	Entity	Title, genre, publisher, price, stock — Encapsulation
Order / OrderDetail	Entity	A purchase and its line items
Payment	Entity	Amount, method, status for an order

Class	Type	Responsibility / OOP role
DatabaseConnection	Data access	Central ADO.NET connection & command runner — Abstraction
GameService / OrderService	Service	CRUD + business logic between forms and the database

7. Database Schema Diagram

7.1 Core Schema

The core relational schema below is normalized to 3NF. Primary keys (PK) and foreign keys (FK) are marked; separate Genres and Publishers tables remove transitive dependencies from the Games table, and OrderDetails resolves the many-to-many relationship between Orders and Games.

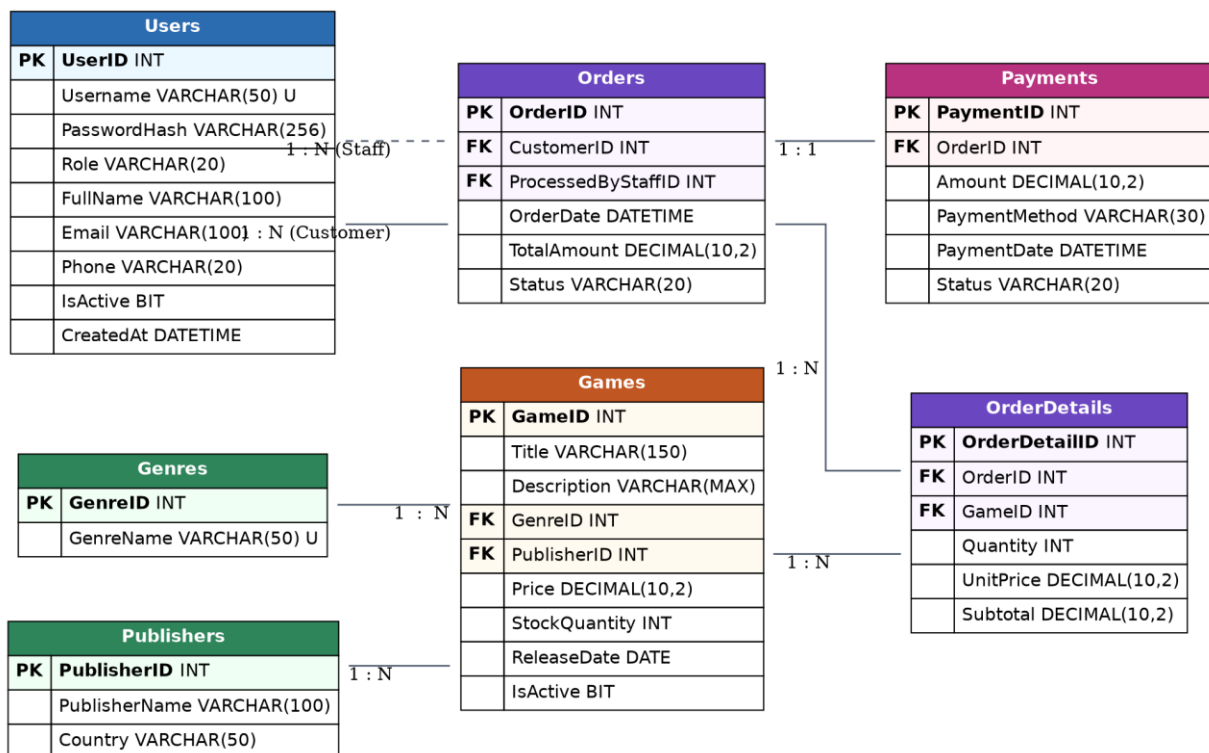


Figure 3: Core Entity-Relationship / Database Schema Diagram (3NF)

Key Relationships

- Genres 1 : N Games and Publishers 1 : N Games — each game has one genre and one publisher.
- Users 1 : N Orders — a customer places many orders; a staff member processes many orders.
- Orders 1 : N OrderDetails and Games 1 : N OrderDetails — OrderDetails links orders to games (resolves M:N).
- Orders 1 : 1 Payments — each order has one payment record.

7.2 Extended Schema (Subscriptions, Wallet, Reviews & Wishlist)

The additional tables below support the subscription plans (Section 3) and the unique features (Section 4). They all link back to the existing Users and Games tables and keep the design in 3NF.

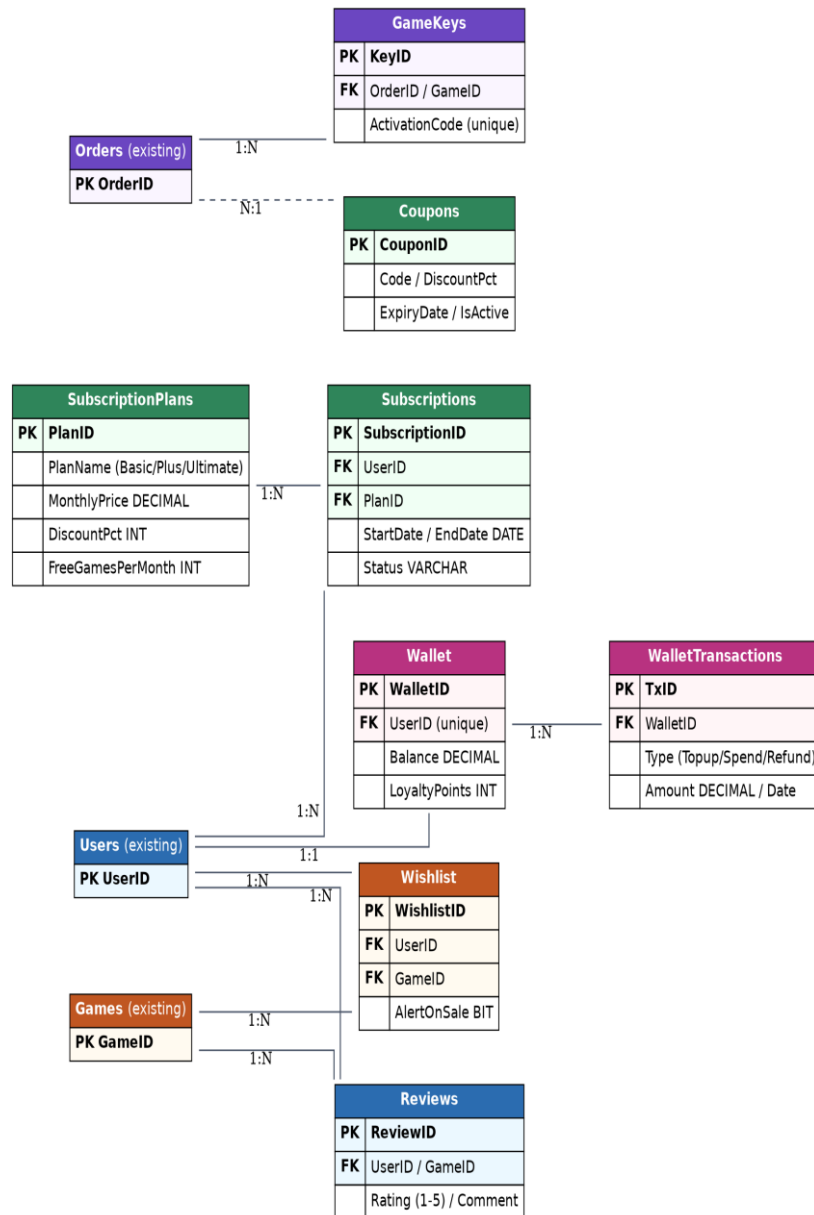


Figure 4: Extended schema for subscription and value-adding features

New Tables & Relationships

- SubscriptionPlans 1 : N Subscriptions and Users 1 : N Subscriptions — a user holds a membership of one plan at a time.
- Users 1 : N Reviews and Games 1 : N Reviews — each review belongs to one user and one game.
- Users 1 : N Wishlist and Games 1 : N Wishlist — a user can wishlist many games.
- Users 1 : N WalletTransactions — every top-up and spend is recorded as one transaction.

7.3 Complete ER Diagram (All 17 Tables)

The consolidated diagram below combines the core and extended schemas into a single reference view, showing every table in GameHubDB together with primary keys (PK), foreign keys (FK), and the enforced/logical relationships between them.

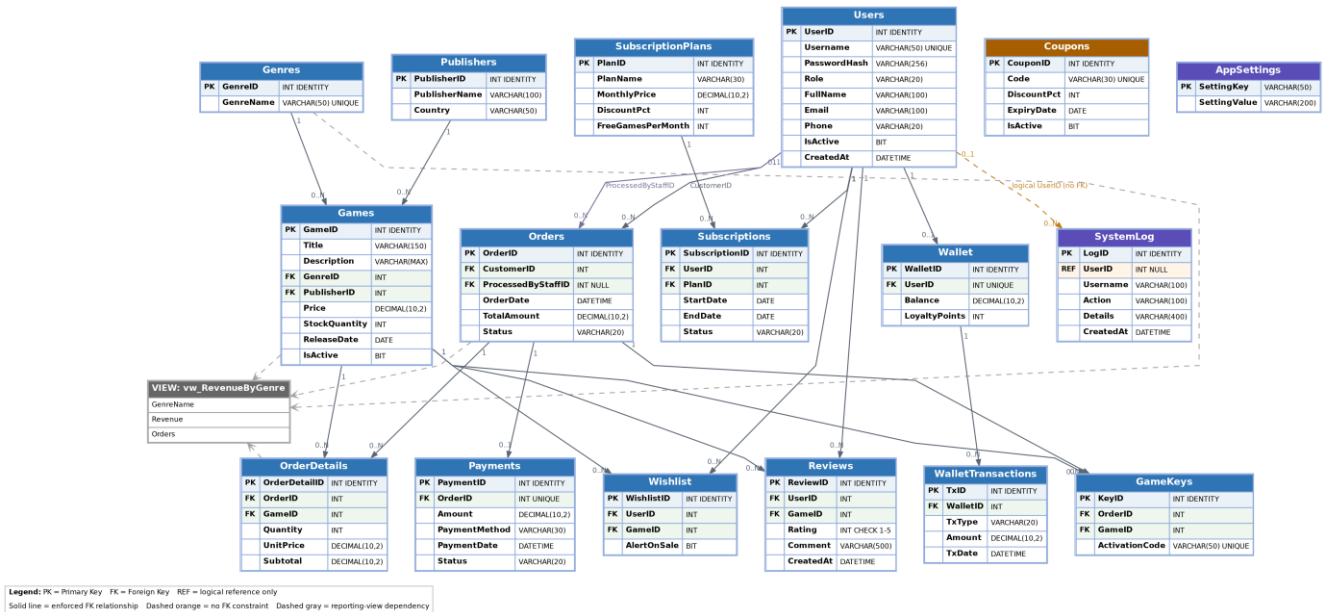


Figure 5: Complete Entity-Relationship diagram of the GameHubDB database (17 tables + reporting view)

8. Database Implementation (SQL Queries)

This section lists the actual SQL Server (T-SQL) scripts used to build and operate the database. The DDL creates every table shown in the schema diagrams; the DML scripts show the CRUD, authentication, and reporting queries the application runs through its DatabaseConnection class.

8.1 Database & Core Tables (DDL)

```
CREATE DATABASE GameHubDB;
GO
USE GameHubDB;
GO

CREATE TABLE Users (
    UserID          INT IDENTITY(1,1) PRIMARY KEY,
    Username        VARCHAR(50) NOT NULL UNIQUE,
    PasswordHash    VARCHAR(256) NOT NULL,
    Role            VARCHAR(20) NOT NULL,          -- Admin / Staff / Customer
    FullName        VARCHAR(100) NOT NULL,
    Email           VARCHAR(100),
    Phone           VARCHAR(20),
    IsActive        BIT NOT NULL DEFAULT 1,
    CreatedAt       DATETIME NOT NULL DEFAULT GETDATE()
);

CREATE TABLE Genres (
    GenreID INT IDENTITY(1,1) PRIMARY KEY,
    GenreName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Publishers (
    PublisherID INT IDENTITY(1,1) PRIMARY KEY,
    PublisherName VARCHAR(100) NOT NULL,
    Country      VARCHAR(50)
);

CREATE TABLE Games (
    GameID          INT IDENTITY(1,1) PRIMARY KEY,
    Title           VARCHAR(150) NOT NULL,
    Description      VARCHAR(MAX),
    GenreID         INT NOT NULL FOREIGN KEY REFERENCES Genres(GenreID),
    PublisherID     INT NOT NULL FOREIGN KEY REFERENCES Publishers(PublisherID),
    Price           DECIMAL(10,2) NOT NULL,
    StockQuantity   INT NOT NULL DEFAULT 0,
    ReleaseDate     DATE,
    IsActive        BIT NOT NULL DEFAULT 1
);

CREATE TABLE Orders (
    OrderID         INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID      INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    ProcessedByStaffID INT NULL FOREIGN KEY REFERENCES Users(UserID),
    OrderDate       DATETIME NOT NULL DEFAULT GETDATE(),
    TotalAmount     DECIMAL(10,2) NOT NULL,
    Status          VARCHAR(20) NOT NULL DEFAULT 'Pending'
);

CREATE TABLE OrderDetails (
    OrderDetailID INT IDENTITY(1,1) PRIMARY KEY,
    OrderID       INT NOT NULL FOREIGN KEY REFERENCES Orders(OrderID),
    GameID        INT NOT NULL FOREIGN KEY REFERENCES Games(GameID),
    Quantity      INT NOT NULL,
    UnitPrice     DECIMAL(10,2) NOT NULL,
    Subtotal      DECIMAL(10,2) NOT NULL
);

CREATE TABLE Payments (
    PaymentID      INT IDENTITY(1,1) PRIMARY KEY,
    OrderID        INT NOT NULL UNIQUE FOREIGN KEY REFERENCES Orders(OrderID),
    Amount         DECIMAL(10,2) NOT NULL,
```

```

    PaymentMethod VARCHAR(30),
    PaymentDate   DATETIME NOT NULL DEFAULT GETDATE(),
    Status        VARCHAR(20) NOT NULL DEFAULT 'Completed'
);

```

Script 1: Creates the database and the seven core tables (3NF).

8.2 Extended Feature Tables (DDL)

These tables support the subscription plans (Section 3) and the value-adding features (Section 4).

```

CREATE TABLE SubscriptionPlans (
    PlanID          INT IDENTITY(1,1) PRIMARY KEY,
    PlanName        VARCHAR(30) NOT NULL,          -- Basic / Plus / Ultimate
    MonthlyPrice    DECIMAL(10,2) NOT NULL,
    DiscountPct     INT NOT NULL DEFAULT 0,
    FreeGamesPerMonth INT NOT NULL DEFAULT 0
);

CREATE TABLE Subscriptions (
    SubscriptionID  INT IDENTITY(1,1) PRIMARY KEY,
    UserID         INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    PlanID         INT NOT NULL FOREIGN KEY REFERENCES SubscriptionPlans(PlanID),
    StartDate      DATE NOT NULL DEFAULT GETDATE(),
    EndDate        DATE NOT NULL,
    Status         VARCHAR(20) NOT NULL DEFAULT 'Active'
);

CREATE TABLE Wallet (
    WalletID       INT IDENTITY(1,1) PRIMARY KEY,
    UserID         INT NOT NULL UNIQUE FOREIGN KEY REFERENCES Users(UserID),
    Balance        DECIMAL(10,2) NOT NULL DEFAULT 0,
    LoyaltyPoints  INT NOT NULL DEFAULT 0
);

CREATE TABLE WalletTransactions (
    TxID           INT IDENTITY(1,1) PRIMARY KEY,
    WalletID       INT NOT NULL FOREIGN KEY REFERENCES Wallet(WalletID),
    TxType         VARCHAR(20) NOT NULL,           -- Topup / Spend / Refund
    Amount         DECIMAL(10,2) NOT NULL,
    TxDate         DATETIME NOT NULL DEFAULT GETDATE()
);

CREATE TABLE Wishlist (
    WishlistID     INT IDENTITY(1,1) PRIMARY KEY,
    UserID         INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    GameID         INT NOT NULL FOREIGN KEY REFERENCES Games(GameID),
    AlertOnSale    BIT NOT NULL DEFAULT 1
);

CREATE TABLE Reviews (
    ReviewID      INT IDENTITY(1,1) PRIMARY KEY,
    UserID        INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),
    GameID        INT NOT NULL FOREIGN KEY REFERENCES Games(GameID),
    Rating        INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),
    Comment       VARCHAR(500),
    CreatedAt     DATETIME NOT NULL DEFAULT GETDATE()
);

CREATE TABLE GameKeys (
    KeyID         INT IDENTITY(1,1) PRIMARY KEY,
    OrderID       INT NOT NULL FOREIGN KEY REFERENCES Orders(OrderID),
    GameID        INT NOT NULL FOREIGN KEY REFERENCES Games(GameID),
    ActivationCode VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Coupons (
    CouponID      INT IDENTITY(1,1) PRIMARY KEY,
    Code          VARCHAR(30) NOT NULL UNIQUE,
    DiscountPct   INT NOT NULL,
    ExpiryDate    DATE,
    IsActive      BIT NOT NULL DEFAULT 1
);

```

```
);
```

Script 2: Subscription, wallet, wishlist, review, key and coupon tables.

8.3 CRUD Operations (Games example)

The four CRUD operations required by the course, shown on the Games table. Parameters (@search, etc.) are passed safely as SqlParameterers to prevent SQL injection.

```
-- CREATE : add a new game
INSERT INTO Games (Title, Description, GenreID, PublisherID, Price, StockQuantity, ReleaseDate)
VALUES ('Elden Ring', 'Open-world action RPG', 1, 1, 3999.00, 42, '2022-02-25');

-- READ : list active games with search (by title or genre)
SELECT g.GameID, g.Title, ge.GenreName, g.Price, g.StockQuantity
FROM Games g
JOIN Genres ge ON g.GenreID = ge.GenreID
WHERE g.IsActive = 1
      AND (g.Title LIKE '%' + @search + '%' OR ge.GenreName LIKE '%' + @search + '%');

-- UPDATE : change price and stock
UPDATE Games SET Price = 2999.00, StockQuantity = 30 WHERE GameID = 1;

-- DELETE : soft delete (Staff can only deactivate, not remove)
UPDATE Games SET IsActive = 0 WHERE GameID = 1;
```

Script 3: Create, Read (search), Update, and soft-Delete on Games.

8.4 Authentication & Reporting Queries

```
-- LOGIN : validate an active user (ID + Password only)
SELECT UserID, Role, FullName
FROM Users
WHERE Username = @id AND PasswordHash = @pwd AND IsActive = 1;

-- MEMBER DISCOUNT : fetch active subscription discount at checkout
SELECT sp.DiscountPct
FROM Subscriptions s
JOIN SubscriptionPlans sp ON s.PlanID = sp.PlanID
WHERE s.UserID = @userId AND s.Status = 'Active' AND s.EndDate >= GETDATE();

-- REPORT : revenue by genre (used by the Admin Reports screen)
CREATE VIEW vw_RevenueByGenre AS
SELECT ge.GenreName,
       SUM(od.Subtotal) AS Revenue,
       COUNT(DISTINCT o.OrderID) AS Orders
FROM OrderDetails od
JOIN Games g ON od.GameID = g.GameID
JOIN Genres ge ON g.GenreID = ge.GenreID
JOIN Orders o ON od.OrderID = o.OrderID
WHERE o.Status = 'Paid'
GROUP BY ge.GenreName;
```

Script 4: Login validation, member-discount lookup, and the revenue report view.

8.5 Live Database Verification (SSMS)

The schema above was executed against a live Microsoft SQL Server instance. The screenshots below confirm the GameHubDB database, all 17 tables, and sample seeded data as they exist in SQL Server Management Studio (SSMS).

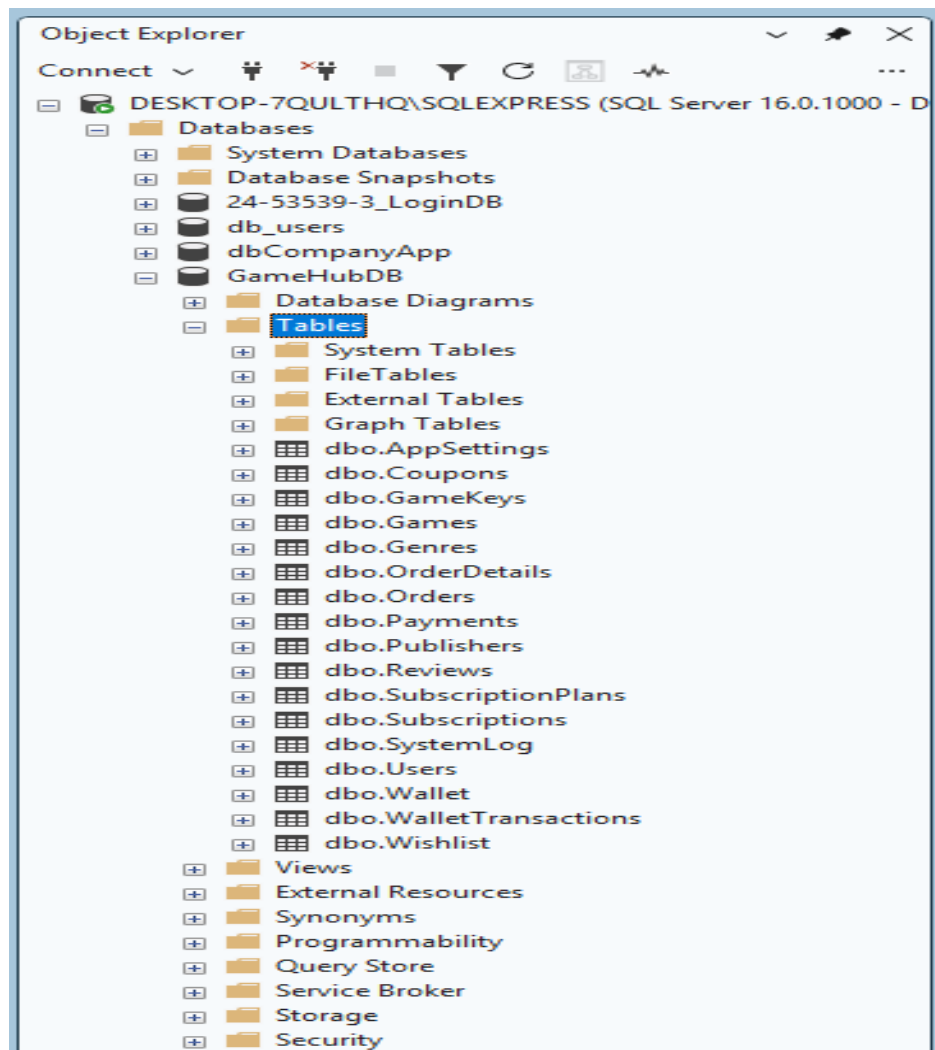


Figure 6: SQL Server Object Explorer showing the GameHubDB database and its tables

[illegible]

Figure 7: Sample SELECT queries confirming seeded data across the core and extended tables

9. Navigation Structure

Navigation starts at a single Login screen. After authentication the user is routed to the dashboard for their role, and every screen is reachable through a persistent navigation bar (a sidebar menu on Admin/Staff forms and a top navigation bar on the Customer store). A Logout option is always available. The map below shows how all forms connect.

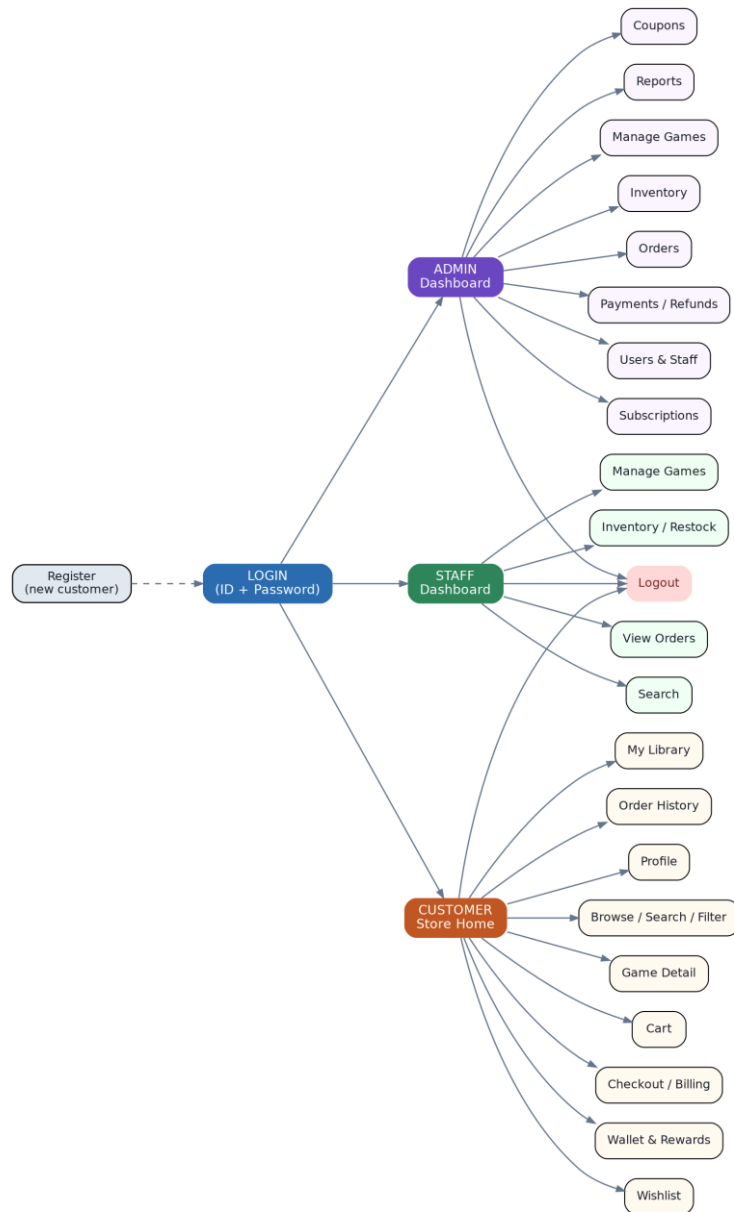


Figure 8: Application navigation map (all forms and role menus)

9.1 Role-Based Navigation Bars

Each role sees only the menu items it is allowed to use, enforcing the access rules from Section 2.

Role	Navigation Bar Items
Admin (sidebar)	Dashboard Manage Games Inventory Orders Payments & Refunds Users & Staff Subscriptions Coupons Reports Logout
Staff (sidebar)	Dashboard Manage Games Inventory / Restock View Orders Search Logout
Customer (top bar)	Store Home Search & Filter Game Detail Cart Checkout Wallet & Rewards Wishlist My Library Order History Profile Logout

Implementation note: the navigation bar is built once as a reusable panel/menu and shown on every form, so navigation is consistent and the code stays modular (a single place to add or restrict menu items per role).

10. Output Screenshots

The following screens are captured directly from the running GameHub C# WinForms application connected to the live SQL Server database.

10.1 Registration

New customers sign up with full name, username, email, phone, and password; every public registration is created with the Customer role.

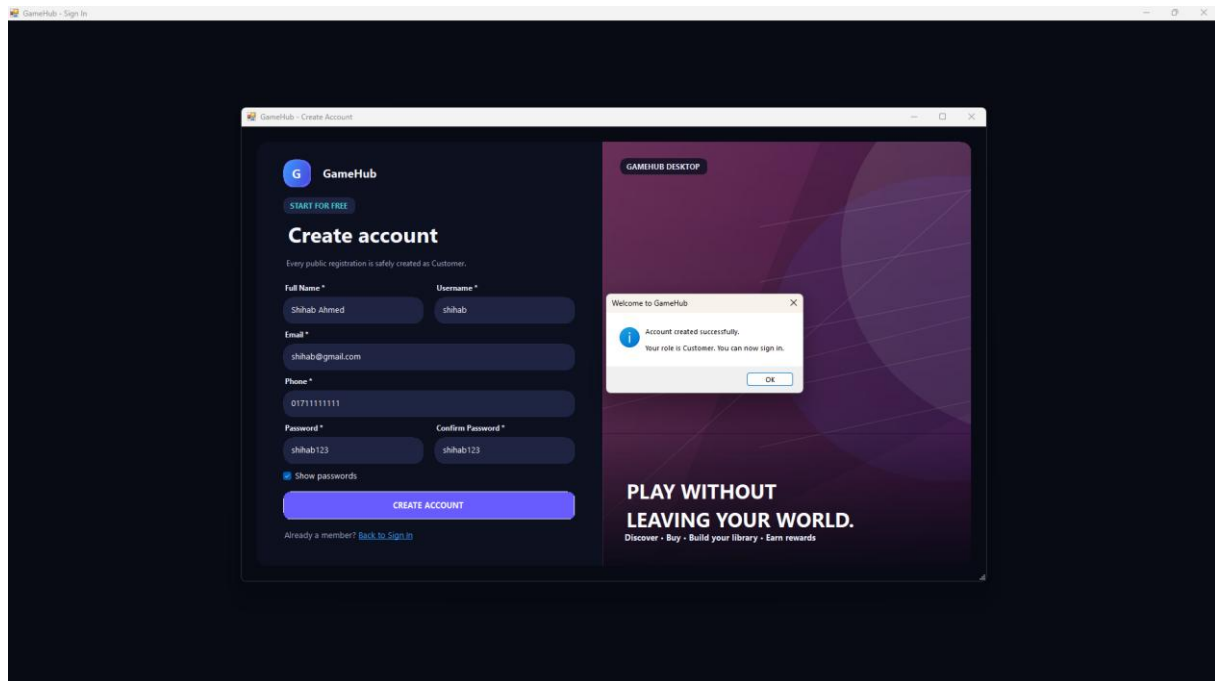
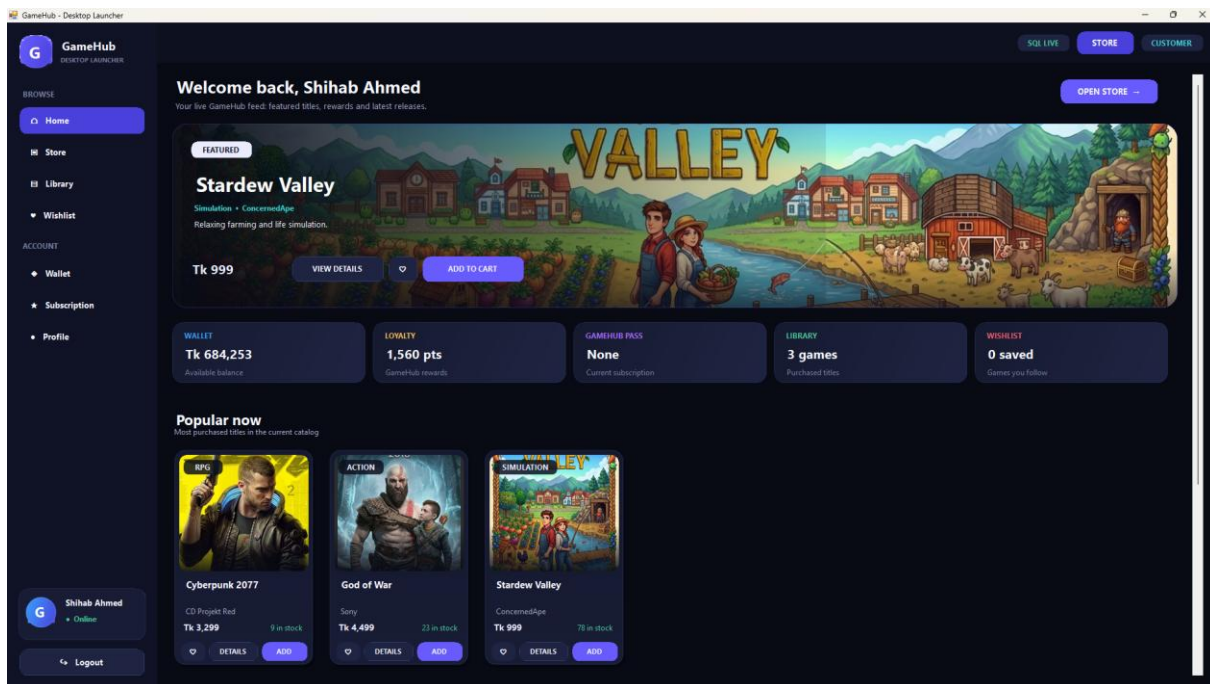


Figure 9: Registration / create-account screen

10.2 Login-Routed Home (Customer)

Landing screen after a customer signs in, showing wallet balance, loyalty points, active subscription, library



count, wishlist count, and popular titles.

Figure 10: Customer home screen after login

10.3 Admin Dashboard

Landing screen for the Admin, with summary cards (games, orders, revenue, active users), a sidebar for navigation, and a recent-orders grid.

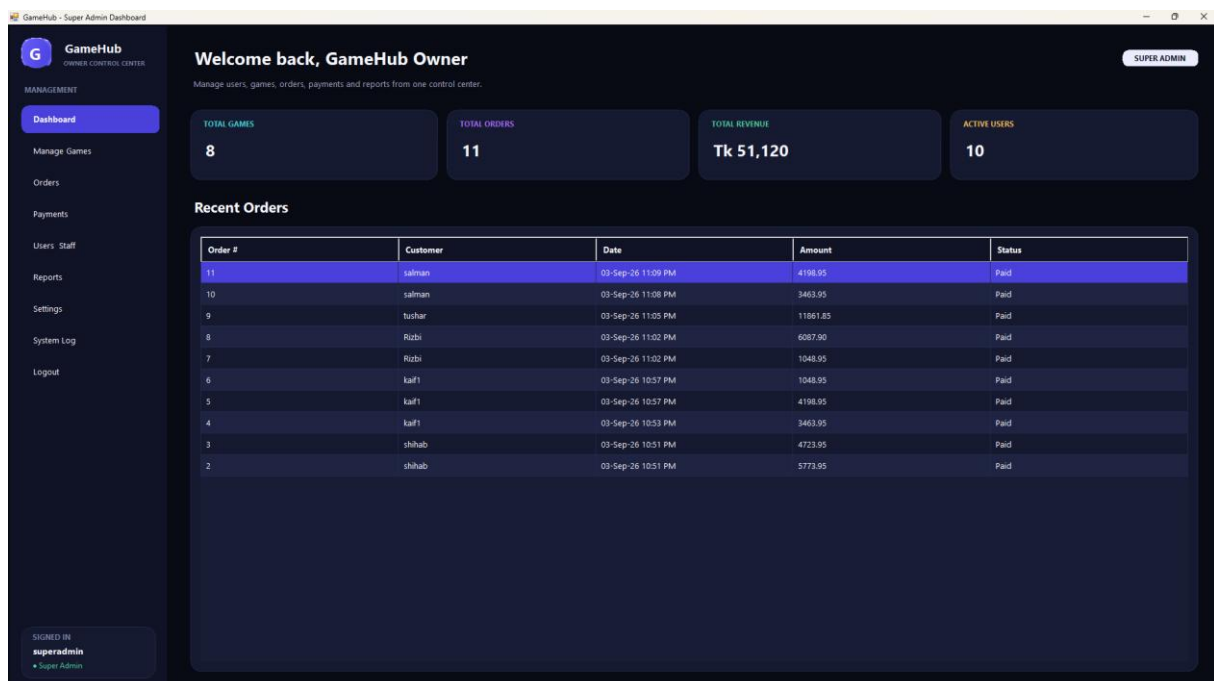


Figure 11: Admin dashboard with KPI cards and recent orders

10.4 Manage Games (CRUD)

Demonstrates Create, Read, Update, and Delete on the Games table, with a search bar and a catalog grid bound to the database.

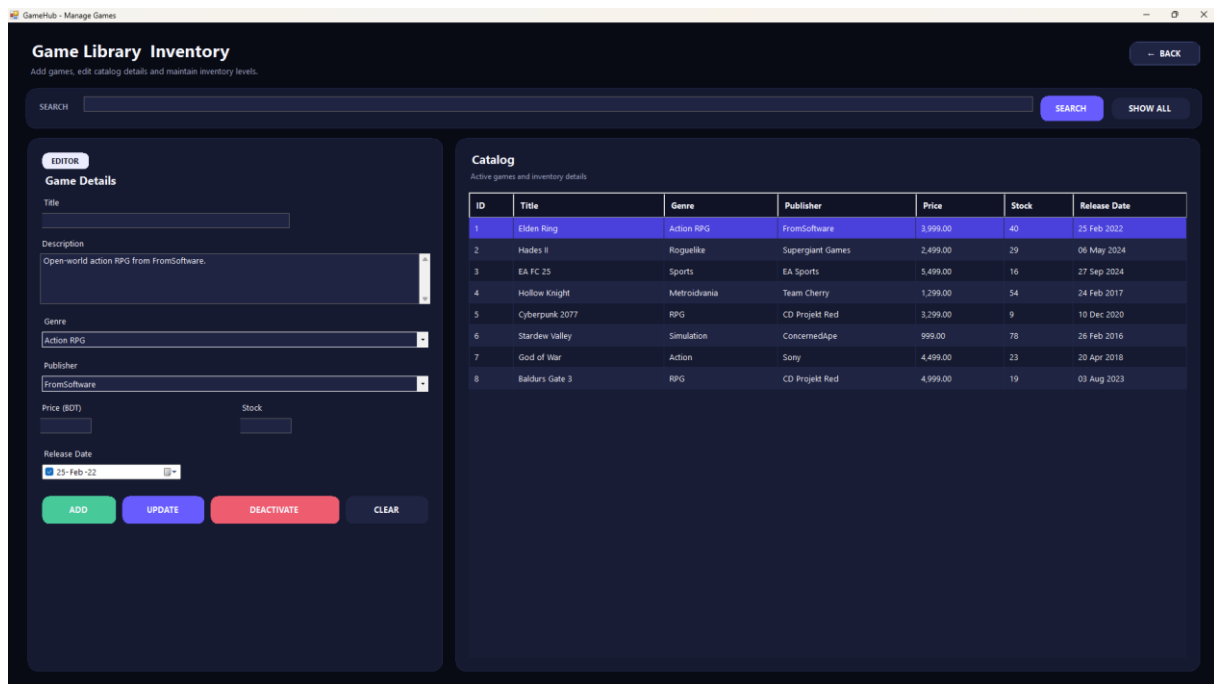


Figure 12: Manage Games — full CRUD with search

10.5 Customer Store / Browse

Customer-facing store grid with search, genre filter, a featured banner, and an Add-to-cart button on every card.

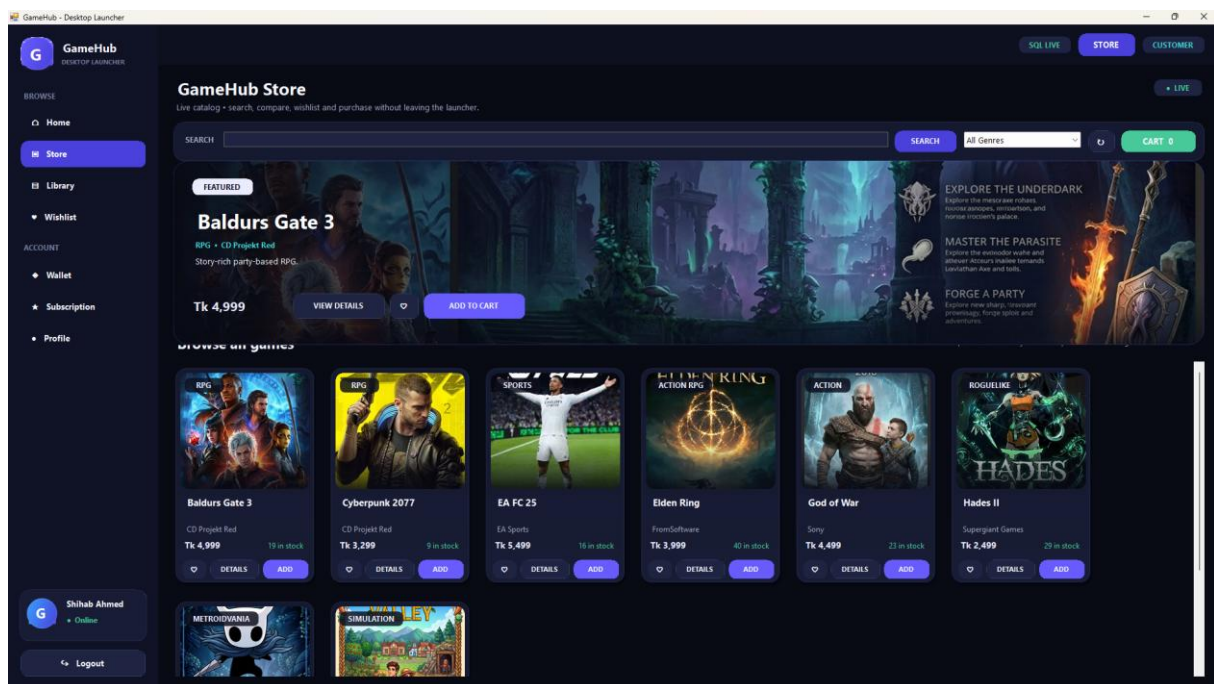


Figure 13: Customer store — browse, search & filter by genre

10.6 My Library

Shows every game a customer owns after purchase, with quick access to view or launch details.

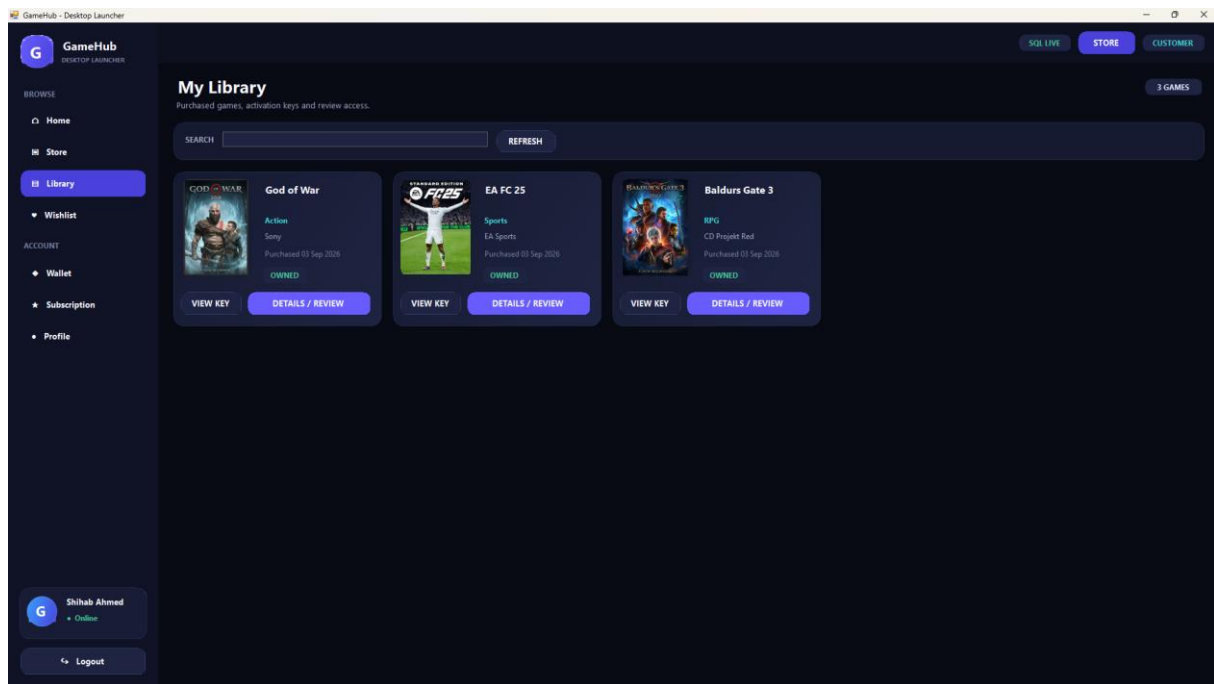


Figure 14: My Library — owned games after purchase

10.7 Wishlist

Customers save games they are interested in and can be alerted when the price drops or the game goes on sale.

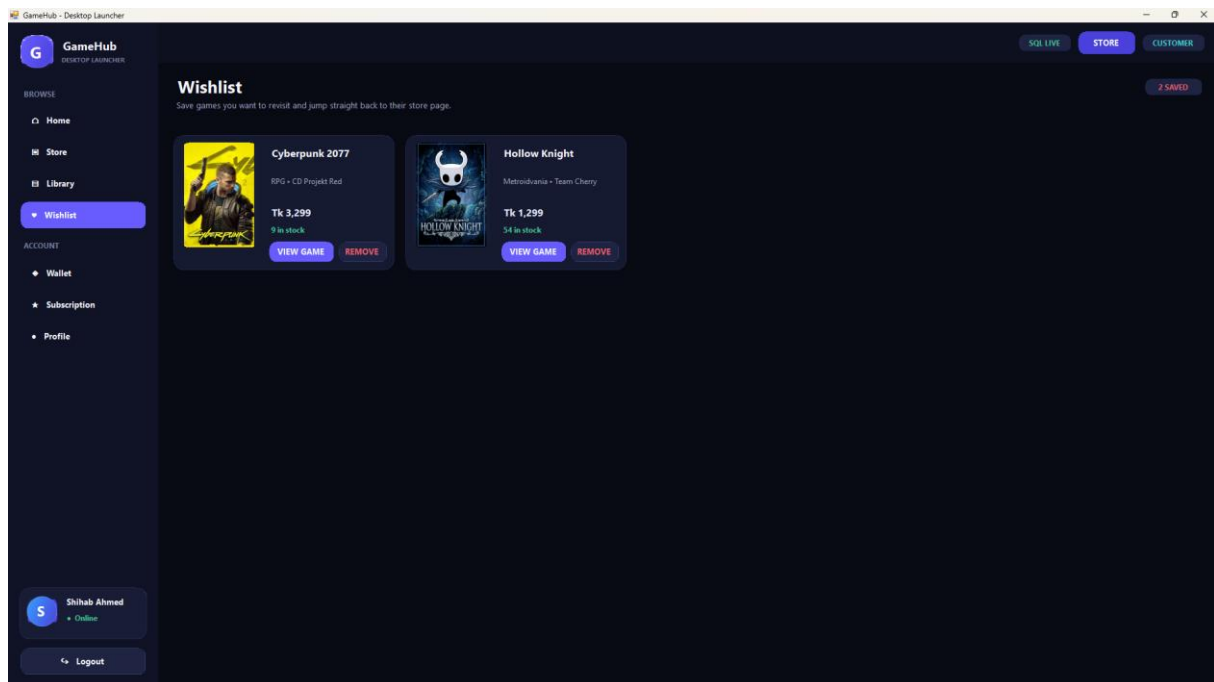


Figure 15: Wishlist screen with saved games

10.8 Wallet & Loyalty Rewards

Customers top up a prepaid balance and spend it at checkout; every credit and debit is logged with loyalty points shown alongside.

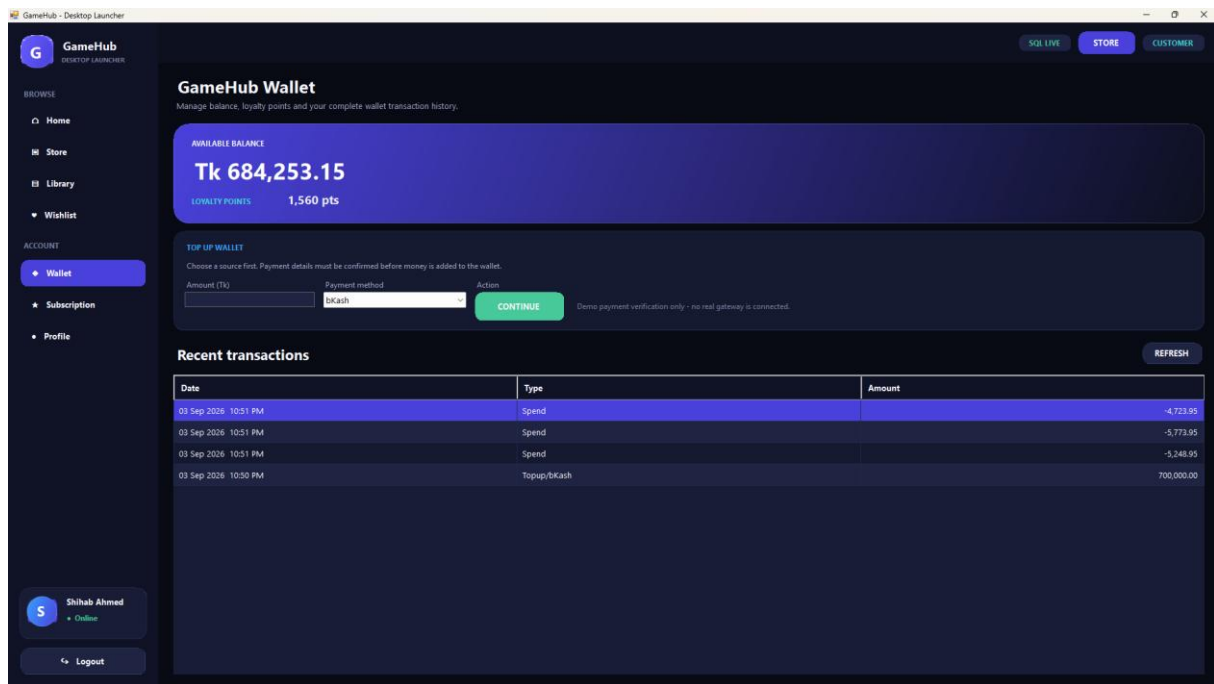


Figure 16: GameHub Wallet screen with balance, loyalty points & transaction history

10.9 Subscription Plans (GameHub Pass)

Customers pick a membership tier for recurring discounts, free monthly games, and early access to sales.

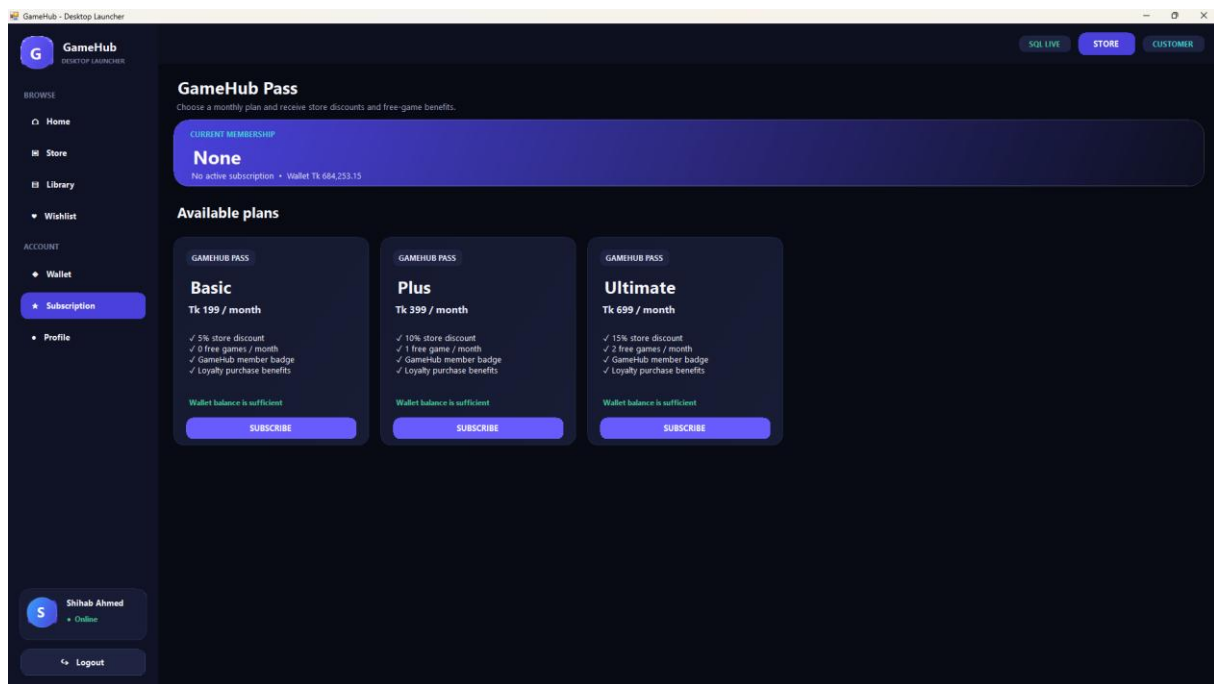


Figure 17: Subscription plans screen with tiered membership cards

10.10 Checkout & Billing

Generates an itemized invoice, lets the customer pick a payment method, applies VAT, and shows the order summary before confirming.

GameHub - Checkout

Checkout
Review your order, choose a payment method and confirm the purchase.

INVOICE Customer: shihab • 03 Sep 2026

Game	Qty	Unit Price	Subtotal
Hollow Knight	1	Tk 1,299.00	Tk 1,299.00

PAYMENT DETAILS

Payment method

- Wallet
- bkash
- Nagad
- Credit / Debit Card
- Bank Transfer

ORDER SUMMARY

Subtotal Tk 1,299.00
Membership 0% - Tk 0.00
VAT 5% Tk 64.95
TOTAL Tk 1,363.95

You will be asked to confirm before the order is created.

[← BACK](#) [CONFIRM PURCHASE](#)

Figure 18: Checkout & billing with itemized invoice

10.11 Purchase Confirmation

Confirms a completed purchase, issues the activation key, and adds the game directly to the customer's library.

GameHub - Checkout

Checkout
Review your order, choose a payment method and confirm the purchase.

INVOICE Customer: shihab • 03 Sep 2026

Game	Qty	Unit Price	Subtotal
Hollow Knight	1	Tk 1,299.00	Tk 1,299.00

PAYMENT DETAILS

Payment method

- Wallet

Available wallet balance: Tk 684,253.15. The wallet is charged in the same SQL transaction as the order.

ORDER SUMMARY

Subtotal Tk 1,299.00
Membership 0% - Tk 0.00
VAT 5% Tk 64.95
TOTAL Tk 1,363.95

You will be asked to confirm before the order is created.

[← BACK](#) [CONFIRM PURCHASE](#)

Purchase complete

PURCHASE SUCCESSFUL

Order #: 12
Status: Paid
Payment: Wallet
Total: Tk 1,363.95

Activation keys:
Hollow Knight : ON4YR-ZTHKX-BRXQB

The purchased game is now available in your Library.

[OK](#)

Figure 19: Purchase-complete confirmation with activation key

10.12 Order Management (Admin/Staff)

Lists every order with customer, date, total, and status, and lets staff drill into an order's line items.

Order Management
Review customer orders, inspect items and update order status.

SEARCH SHOW ALL

Orders

Order ID	Customer	Customer Name	Date	Total	Status	Processed By
11	salman	Salman	03-Sep-26 11:09 PM	4198.95	Paid	-
10	salman	Salman	03-Sep-26 11:08 PM	3463.95	Paid	-
9	tushar	Tanvir Tushar	03-Sep-26 11:05 PM	11861.85	Paid	-
8	Rizbi	Mahfuj Rizbi	03-Sep-26 11:02 PM	6087.90	Paid	-
7	Rizbi	Mahfuj Rizbi	03-Sep-26 11:02 PM	1048.95	Paid	-
6	kaif	kaif	03-Sep-26 10:57 PM	1048.95	Paid	-
5	kaif	kaif	03-Sep-26 10:57 PM	4198.95	Paid	-
4	kaif	kaif	03-Sep-26 10:53 PM	3463.95	Paid	-
3	shihab	Shihab Ahmed	03-Sep-26 10:51 PM	4723.95	Paid	-
2	shihab	Shihab Ahmed	03-Sep-26 10:51 PM	5773.95	Paid	-
1	shihab	Shihab Ahmed	03-Sep-26 10:51 PM	5248.95	Paid	-

PROCESS

Selected Order
Order #11 • salman

Status
Paid

UPDATE STATUS

Order Items

Game	Qty	Unit Price	Subtotal
Eliden Ring	1	3999.00	3999.00

Figure 20: Order management — order list and line-item detail

10.13 Payment Management (Admin)

Full log of payments across all orders, including method, amount, date, and status for reconciliation.

Payment Management
Review customer payments, transaction methods and payment status.

SEARCH SHOW ALL

PAYMENTS

Payment ID	Order ID	Customer	Amount	Method	Date	Status
11	11	salman	4198.95	Wallet	03-Sep-26 11:09 PM	Completed
10	10	salman	3463.95	bkash	03-Sep-26 11:08 PM	Completed
9	9	tushar	11861.85	Wallet	03-Sep-26 11:05 PM	Completed
8	8	Rizbi	6087.90	Wallet	03-Sep-26 11:02 PM	Completed
7	7	Rizbi	1048.95	Wallet	03-Sep-26 11:02 PM	Completed
6	6	kaif	1048.95	Wallet	03-Sep-26 10:57 PM	Completed
5	5	kaif	4198.95	Wallet	03-Sep-26 10:57 PM	Completed
4	4	kaif	3463.95	bkash	03-Sep-26 10:53 PM	Completed
3	3	shihab	4723.95	Wallet	03-Sep-26 10:51 PM	Completed
2	2	shihab	5773.95	Wallet	03-Sep-26 10:51 PM	Completed
1	1	shihab	5248.95	Wallet	03-Sep-26 10:51 PM	Completed

Figure 21: Payment management and reconciliation view

10.14 Users & Staff

Admin view for creating, editing, activating, and deactivating Staff and Customer accounts.

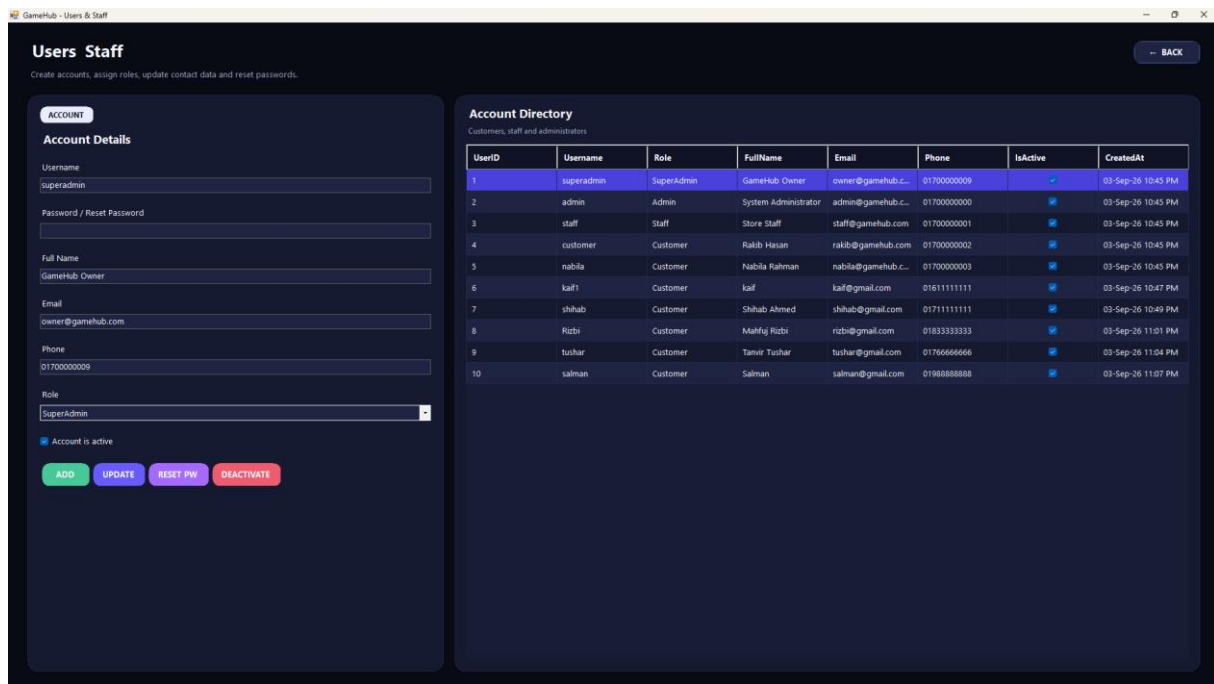


Figure 22: Users & Staff account management

10.15 Player Profile

Lets a customer view and edit their own profile information and change their password.

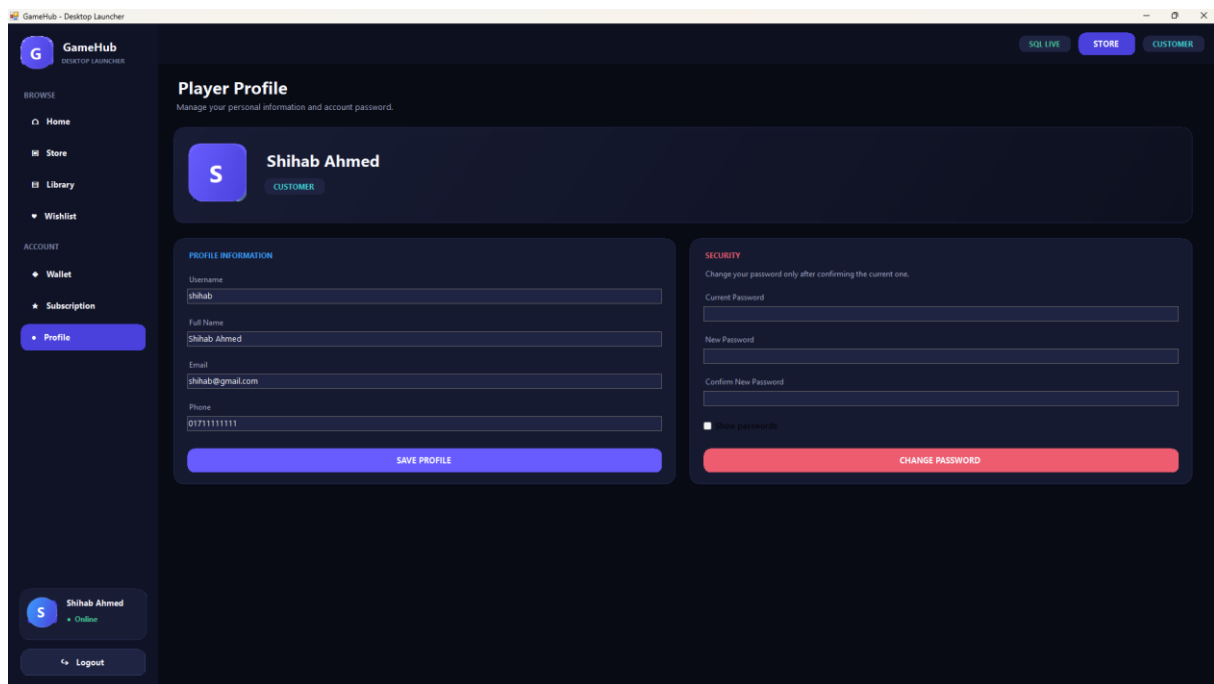


Figure 23: Player profile screen

10.16 Owner Settings

Admin-only settings for shop name, currency symbol, VAT percent, and default discount.

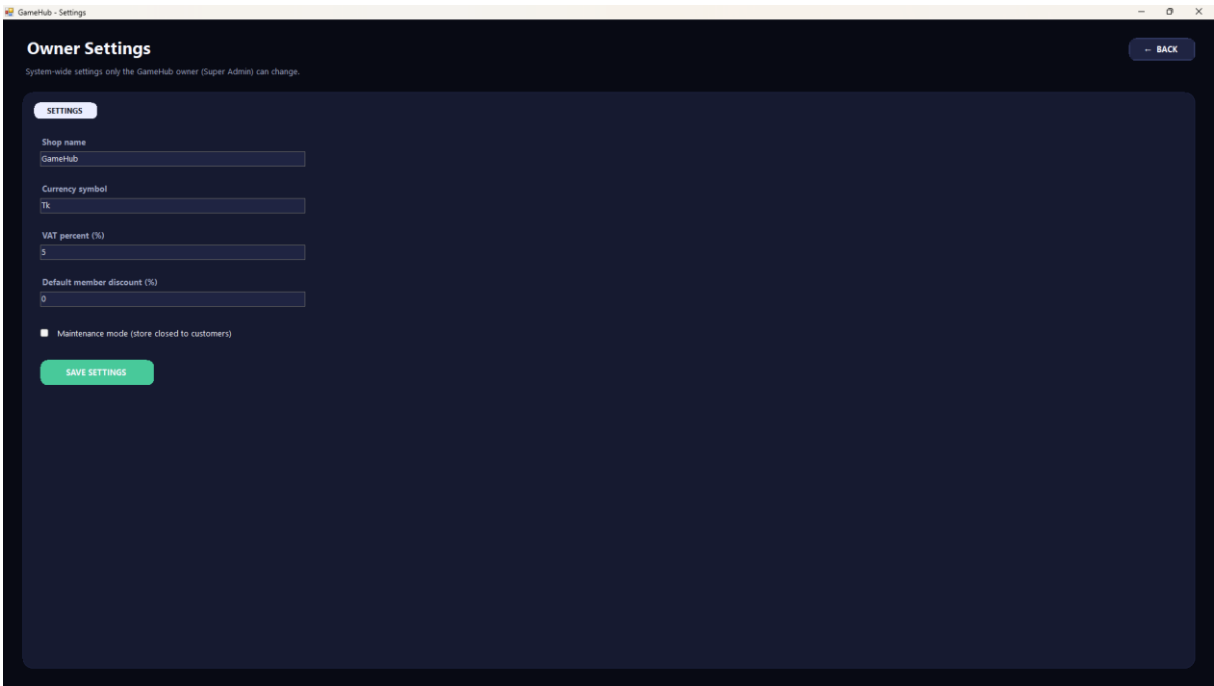


Figure 24: Owner / Admin settings screen

10.17 System Activity Log

Owner-facing audit trail recording key actions (such as logins) performed across GameHub.

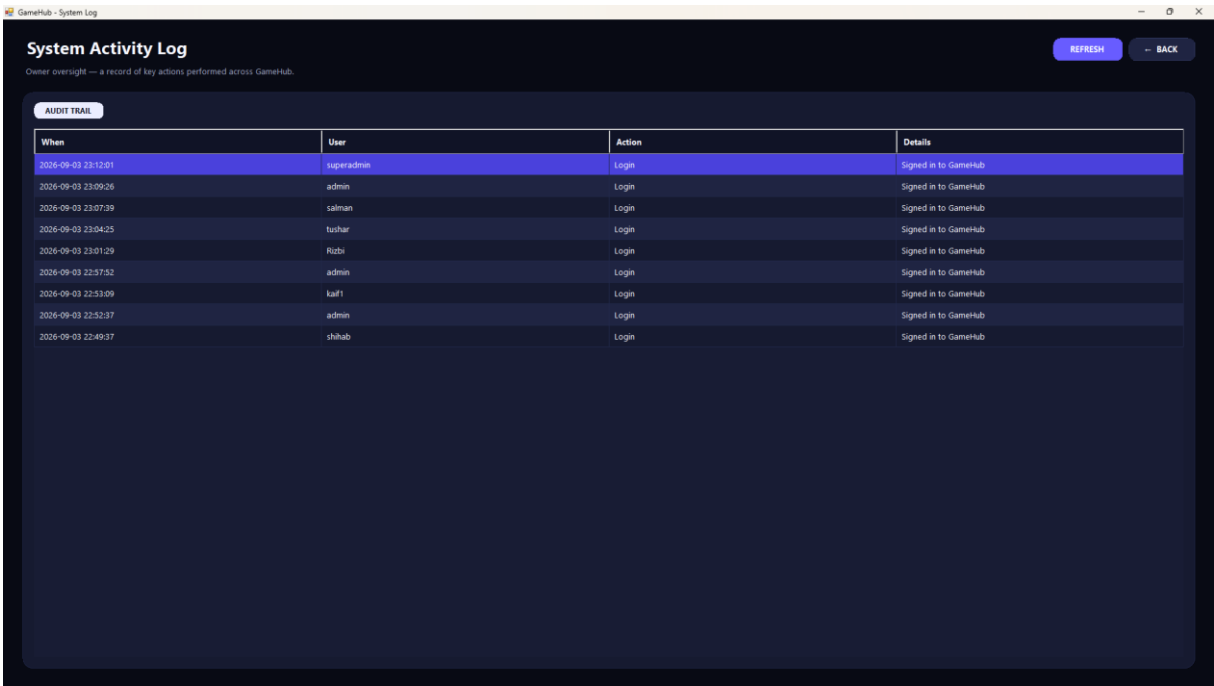


Figure 25: System activity log / audit trail

10.18 Sales & Revenue Reports

Management view: revenue-by-genre chart and a summary panel of key sales figures for the Admin.

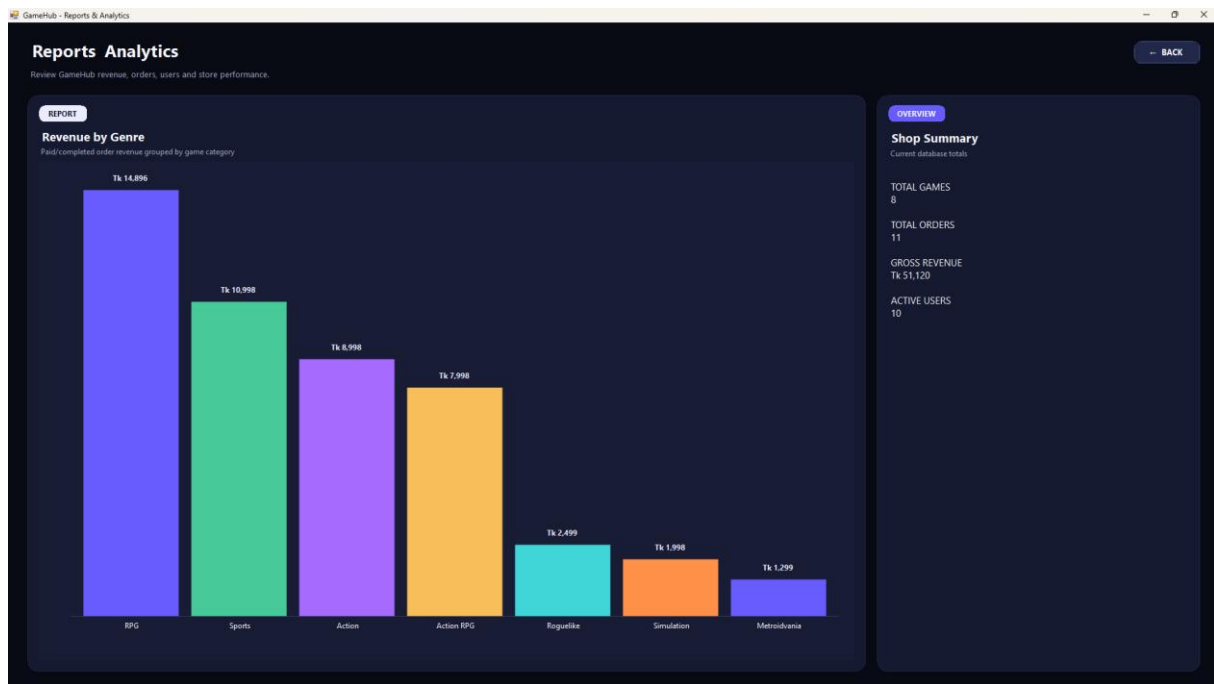


Figure 26: Sales & revenue report (Admin only)

Note: the screens above are captured from the running GameHub application (C# WinForms, connected to Microsoft SQL Server) during functional testing.

11. Tools & Technologies

Area	Technology
Language	C# (.NET)
UI Framework	Windows Forms (WinForms) — Desktop
Database	Microsoft SQL Server (SSMS)
Data Access	ADO.NET (SqlConnection / SqlCommand)
IDE	Visual Studio
Design / Mockups	Figma
Version Control	Git / GitHub

12. Additional Notes

- The project clearly demonstrates all four OOP concepts (encapsulation, inheritance, polymorphism, abstraction) in C#.
- Code readability, structure, and organization are prioritized through a layered, modular design.
- The subscription plans (Section 3), value-adding features (Section 4), and full SQL scripts (Section 8) go beyond the base requirements and are designed to improve the project grade.